

Sprint Backlog — Sprint 3

Music Playlist Manager

Gruppo 04 | Corso: Software Architecture Design | A.A. 2025/2026

Sprint	Data inizio	Data fine	Durata
Sprint 3	11 giugno 2026	18 giugno 2026	1 settimana

Sprint Goal: Completare le funzionalità avanzate del Music Playlist Manager — annullamento delle operazioni di aggiunta e rimozione tramite il pattern Command, visualizzazione delle tracce e playlist più riprodotte, aggiunta di tag visivi alle tracce, generazione automatica di playlist per genere e anno tramite il pattern Factory Method, e riordinamento delle tracce all'interno delle playlist — in modo che l'utente disponga di un controllo completo sulla propria libreria musicale.

Velocity stimata

Parametro	Valore
Sprint precedenti	Sprint 1, Sprint 2
Velocity storica	37 SP (Sprint 1) 54 SP (Sprint 2)
Velocity stimata	44 Story Points $US22: 8 + US23: 5 + US24: 5 + US25: 5 + US26: 8 + US27: 13$
Membri del team	4

Nota: La velocity stimata per lo Sprint 3 è pari a 44 SP, in linea con la velocity media degli sprint precedenti. Lo sprint introduce due nuovi design pattern: **Command** (US22 — Undo delle operazioni) e **Factory Method** (US26 — Generazione automatica di playlist).

Sprint Backlog

DC = Dato che **Q** = Quando **A** = Allora

ID	User Story	Criteri di Accettazione	SP
US22	<i>Come utente</i> voglio poter annullare (undo) qualsiasi operazione di aggiunta o rimozione di una traccia o playlist <i>al fine di</i> correggere errori senza dover rifare manualmente le modifiche.	DC sono nell'applicazione e ho effettuato l'aggiunta di una traccia al catalogo, Q premo il pulsante Undo, A la traccia viene rimossa dal catalogo. DC sono nell'applicazione e ho effettuato l'aggiunta di una playlist alla libreria, Q premo il pulsante Undo, A la playlist viene rimossa dalla libreria. DC sono nell'applicazione e ho effettuato la rimozione di una traccia, Q premo il pulsante Undo, A la traccia viene ripristinata nel catalogo. DC sono nell'applicazione e ho effettuato la rimozione di una playlist dalla libreria, Q premo il pulsante Undo, A la playlist viene ripristinata nella libreria.	8

ID	User Story	Criteri di Accettazione	SP
US23	<i>Come utente</i> voglio poter visualizzare nella home page le tracce riprodotte più frequentemente <i>al fine di</i> ritrovare rapidamente i contenuti che ascolto di più.	• DC sono nell'applicazione, Q apro la home page, A visualizzo una sezione con le tracce ordinate per numero di riproduzioni, dalla più ascoltata alla meno ascoltata. • DC non ho ancora riprodotto nessuna traccia, Q apro la home page, A la sezione "più ascoltate" è vuota o mostra un messaggio "Nessuna riproduzione ancora registrata".	5
US24	<i>Come utente</i> voglio poter visualizzare nella home page le playlist riprodotte più frequentemente <i>al fine di</i> ritrovare rapidamente i contenuti che ascolto di più.	• DC sono nell'applicazione, Q apro la home page, A visualizzo una sezione con le playlist ordinate per numero di riproduzioni, dalla più ascoltata alla meno ascoltata. • DC non ho ancora riprodotto nessuna playlist, Q apro la home page, A la sezione "Più ascoltate" è vuota o mostra un messaggio "Nessuna riproduzione ancora registrata".	5
US25	<i>Come utente</i> voglio poter aggiungere tag visivi alle tracce (preferita, esplicita, nuova uscita) <i>al fine di</i> arricchire la visualizzazione.	• DC sono nell'applicazione e visualizzo una traccia, Q aggiungo uno o più tag (preferita, esplicita, nuova uscita), A il tag viene salvato e appare visivamente accanto alla traccia nel catalogo. • DC sono nell'applicazione e visualizzo una traccia con un tag assegnato, Q rimuovo il tag, A il tag non appare più accanto alla traccia.	5
US26	<i>Come utente</i> voglio che l'applicazione generi automaticamente playlist in base a caratteristiche comuni <i>al fine di</i> avere raggruppamenti di brani senza doverli creare manualmente.	• DC sono nell'applicazione e il catalogo contiene brani appartenenti a diversi generi musicali, Q accedo alla sezione "Playlist Automatiche", A il sistema mostra un elenco di playlist create automaticamente, ognuna corrispondente a un genere specifico presente nel catalogo, contenente esclusivamente i brani associati a quel genere. • DC il catalogo contiene brani pubblicati in anni differenti, Q accedo alla sezione "Playlist Automatiche", A il sistema mostra un elenco di playlist create automaticamente, ognuna corrispondente a un periodo specifico, contenente esclusivamente i brani associati a quel periodo. • DC il sistema ha già generato le playlist automatiche, Q aggiungo un nuovo brano al catalogo, A il sistema inserisce automaticamente il brano nella playlist del genere o dell'anno corrispondente, senza richiedere una rigenerazione manuale. • DC sono nell'applicazione e il catalogo contiene tracce, Q aggiungo uno o più tag ad un brano del catalogo, A il sistema inserisce automaticamente il brano nella playlist automatica associata ai tag assegnati.	8

ID	User Story	Criteri di Accettazione	SP
US27	<i>Come utente</i> voglio poter riordinare le tracce di una playlist in qualsiasi momento, anche durante la riproduzione, <i>al fine di</i> personalizzare l'ordine di ascolto senza interrompere la musica.	• DC sono nell'applicazione e sto riproducendo una traccia da una playlist o dal catalogo, Q sposto una traccia in una posizione diversa nella playlist, A l'ordine aggiornato è visibile immediatamente a schermo senza che la riproduzione si interrompa. • DC sono nell'applicazione e sto riproducendo una traccia da una playlist o dal catalogo, Q riordino le tracce, A la traccia attualmente in riproduzione continua fino alla fine e le tracce successive seguono il nuovo ordine.	13

Riepilogo

User Story	Story Points	Priorità
US22 — Undo delle Operazioni	8	Media
US23 — Frequenza di Riproduzione Tracce	5	Media
US24 — Frequenza di Riproduzione Playlist	5	Media
US25 — Gestione dei Tag	5	Media
US26 — Generazione Automatica di Playlist	8	Media
US27 — Ordinamento della Playlist	13	Bassa
Totale Sprint 3	44 SP	—

Interfacce definite per lo Sprint 3

Si riportano le interfacce introdotte in questo sprint con i relativi metodi, in modo che tutti i membri del team dispongano di un contratto condiviso prima dell'implementazione.

Pattern Command — US22

Interfaccia / Classe	Metodo	Descrizione
Command	<code>void execute()</code>	Esegue l'operazione.
Command	<code>void undo()</code>	Annulla l'operazione eseguita.
CommandManager	<code>void executeCommand(Command)</code>	Esegue il comando e lo inserisce nell' <i>undo stack</i> .
CommandManager	<code>void undo()</code>	Estrae l'ultimo comando dallo stack e ne invoca <code>undo()</code> .
CommandManager	<code>boolean canUndo()</code>	Restituisce <code>true</code> se lo stack non è vuoto.
CommandManager	<code>Command getLastCommand()</code>	Restituisce l'ultimo comando eseguito senza estrarlo.

Concrete Commands: `AddTrackCommand`, `RemoveTrackCommand`, `AddPlaylistCommand`, `RemovePlaylistCommand`, `AddTrackToPlaylistCommand`, `RemoveTrackFromPlaylistCommand`.

Receiver: `TrackController`, `PlaylistController`.

Invoker: `CommandManager`.

Client: i view controller che creano i comandi e li passano al `CommandManager`.

Pattern Factory Method — US26

Interfaccia / Classe	Metodo	Descrizione
<code>AutoPlaylistGenerator</code> (<i>Creator astratto</i>)	<code>Playlist</code> <code>createPlaylist</code> (<code>MusicCatalog</code>)	Metodo factory astratto: ogni sottoclasse crea la playlist secondo il proprio criterio.
<code>AutoPlaylistGenerator</code>	<code>String</code> <code>getCriterionName()</code>	Restituisce il nome del criterio (es. “Rock”, “2020–2024”).
<code>GenrePlaylistGenerator</code> (<i>ConcreteCreator</i>)	<code>Playlist</code> <code>createPlaylist</code> (<code>MusicCatalog</code>)	Filtra le tracce per genere e costruisce la playlist corrispondente.
<code>YearPlaylistGenerator</code> (<i>ConcreteCreator</i>)	<code>Playlist</code> <code>createPlaylist</code> (<code>MusicCatalog</code>)	Filtra le tracce per anno/periodo e costruisce la playlist corrispondente.
<code>TagPlaylistGenerator</code> (<i>ConcreteCreator</i>)	<code>Playlist</code> <code>createPlaylist</code> (<code>MusicCatalog</code>)	Filtra le tracce per tag e costruisce la playlist corrispondente.

Estensioni al Model esistente

Classe	Metodo	Descrizione
<code>Track</code>	<code>void addTag(TagType)</code>	Aggiunge un tag visivo alla traccia.
<code>Track</code>	<code>void</code> <code>removeTag(TagType)</code>	Rimuove un tag visivo dalla traccia.
<code>Track</code>	<code>Set<TagType> getTags()</code>	Restituisce l'insieme dei tag attualmente associati.
<code>Track</code>	<code>boolean</code> <code>hasTag(TagType)</code>	Restituisce <code>true</code> se la traccia ha il tag specificato.
<code>Track</code>	<code>void</code> <code>incrementPlayCount()</code>	Incrementa il contatore di riproduzioni.
<code>Track</code>	<code>int getPlayCount()</code>	Restituisce il numero di riproduzioni della traccia.
<code>Playlist</code>	<code>void</code> <code>incrementPlayCount()</code>	Incrementa il contatore di riproduzioni della playlist.
<code>Playlist</code>	<code>int getPlayCount()</code>	Restituisce il numero di riproduzioni della playlist.
<code>Playlist</code>	<code>boolean moveTrack(int</code> <code>from, int to)</code>	Sposta la traccia dall'indice <code>from</code> all'indice <code>to</code> . Restituisce <code>false</code> se gli indici non sono validi.
<code>TagType</code> (<i>enum</i>)	<code>FAVOURITE,</code> <code>EXPLICIT,</code> <code>NEW_RELEASE</code>	Tipi di tag visivo associabili a una traccia.

Task

Si riportano i task estratti dalle user story presenti nello Sprint Backlog sopra descritto.

#	Categoria	Descrizione
T34	Model	Definire l'interfaccia <code>Command</code> con i metodi <code>execute()</code> e <code>undo()</code> , e implementare i <code>ConcreteCommand</code> : <code>AddTrackCommand</code> , <code>RemoveTrackCommand</code> , <code>AddPlaylistCommand</code> , <code>RemovePlaylistCommand</code> , <code>AddTrackToPlaylistCommand</code> , <code>RemoveTrackFromPlaylistCommand</code> (pattern Command). <i>Prerequisito di T35.</i>
T35	Controller	Implementare <code>CommandManager</code> (Invoker) con i metodi <code>executeCommand(Command)</code> , <code>undo()</code> e <code>canUndo()</code> , basato su uno stack LIFO. Integrare <code>CommandManager</code> con <code>TrackController</code> e <code>PlaylistController</code> sostituendo le chiamate dirette al catalogo con l'esecuzione tramite comandi. <i>Dipende da: T34.</i>
T36	View	Aggiungere il pulsante Undo nella UI (barra superiore o toolbar) e collegarlo al metodo <code>undo()</code> di <code>CommandManager</code> . Il bottone deve essere disabilitato quando <code>canUndo()</code> è <code>false</code> . <i>Dipende da: T35.</i>
T37	Model	Aggiungere a <code>TrackImpl</code> e <code>PlaylistImpl</code> i metodi <code>incrementPlayCount()</code> e <code>getPlayCount()</code> per il conteggio delle riproduzioni. Aggiornare <code>PlaybackController</code> per incrementare i contatori ad ogni avvio di riproduzione. <i>Prerequisito di T38.</i>
T38	View	Implementare nella home page le sezioni "Tracce più ascoltate" e "Playlist più ascoltate", ordinate per <code>getPlayCount()</code> in ordine decrescente, registrandosi come <code>CatalogObserver</code> per aggiornarsi in tempo reale. <i>Dipende da: T37.</i>
T39	Model	Aggiungere a <code>TrackImpl</code> i metodi <code>addTag(TagType)</code> , <code>removeTag(TagType)</code> , <code>getTags()</code> e <code>hasTag(TagType)</code> con il relativo enum <code>TagType</code> (<code>FAVOURITE</code> , <code>EXPLICIT</code> , <code>NEW_RELEASE</code>). Aggiornare la persistenza per includere i tag nel salvataggio. <i>Prerequisito di T40.</i>
T40	View	Mostrare i tag visivi accanto alle tracce nel catalogo e nel dettaglio playlist. Aggiungere nel form di modifica traccia i controlli per aggiungere e rimuovere tag. <i>Dipende da: T39.</i>
T41	Model	Definire la classe astratta <code>AutoPlaylistGenerator</code> (Creator) con il metodo <code>factory</code> astratto <code>createPlaylist(MusicCatalog)</code> e implementare i <code>ConcreteCreator</code> : <code>GenrePlaylistGenerator</code> , <code>YearPlaylistGenerator</code> , <code>TagPlaylistGenerator</code> (pattern Factory Method). <i>Prerequisito di T42.</i>
T42	Controller & View	Implementare la sezione "Playlist Automatiche" nella UI: invocare i generatori sul catalogo corrente, mostrare le playlist generate e aggiornarle automaticamente tramite <code>CatalogObserver</code> quando vengono aggiunte tracce o tag. <i>Dipende da: T41.</i>

#	Categoria	Descrizione
T43	Model	Aggiungere a <code>PlaylistImpl</code> il metodo <code>moveTrack(int from, int to)</code> per il riordinamento delle tracce. Notificare gli observer tramite l'evento <code>PLAYLIST_REORDERED</code> già presente in <code>CatalogEventType</code> . <i>Prerequisito di T44.</i>
T44	View	Implementare il drag-and-drop (o i pulsanti su/giù) per il riordinamento delle tracce nel <code>PlaylistDetailViewController</code> , garantendo che la traccia in riproduzione non venga interrotta e che le tracce successive seguano il nuovo ordine. <i>Dipende da: T43.</i>
T45	Test	Scrivere Unit Test per il pattern Command: verifica di <code>execute()</code> , <code>undo()</code> per ciascun <code>ConcreteCommand</code> , e verifica dello stack in <code>CommandManager</code> (<code>canUndo</code> , sequenze undo multiple). <i>Dipende da: T34, T35.</i>
T46	Test	Scrivere Unit Test per <code>AutoPlaylistGenerator</code> : verifica che ciascun generatore produca playlist contenenti esclusivamente le tracce corrispondenti al criterio (genere, anno, tag), e che l'aggiornamento automatico funzioni correttamente all'aggiunta di nuove tracce. <i>Dipende da: T41.</i>
T47	View	Implementare un pannello di aiuto contestuale (tooltip o sezione dedicata nella UI) che mostri all'utente suggerimenti dinamici in base alla schermata corrente e alla modalità di riproduzione attiva. Esempi: nel catalogo tracce spiegare l'effetto di "Riproduci ora" in modalità sequenziale, shuffle e loop; nel dettaglio playlist descrivere il comportamento di Skip Traccia e Skip Playlist; nella barra di riproduzione illustrare la differenza tra le tre modalità. I suggerimenti si aggiornano automaticamente al cambio di modalità tramite il pattern Observer già esistente (evento <code>STRATEGY_CHANGED</code>).

Suddivisione dei Task

Persona	Pacchetto	Ambito	Task
Gioia	M1	Command Model + Riordinamento Model	T34, T43, T47
Annamaria	M2	Factory Method + Test Factory	T41, T46
Luigi	M3	CommandManager + Undo UI + Test Command	T35, T36, T45
Francesco	M4	Tag + Contatori + View avanzate	T37, T38, T39, T40, T42, T44

Burndown chart

Sprint 3

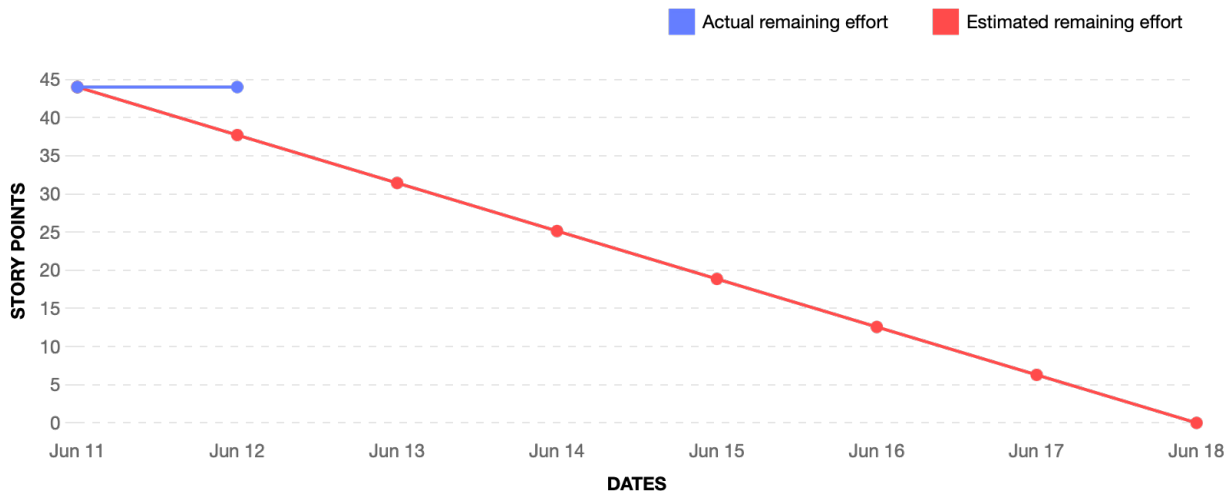


Figura 1: Burndown Chart Sprint 3